TMS — Transport Aggregator

System Design & Architecture

Version 1.0 · Engineering Handover Document
Next.js 14 · Prisma · SQLite/PostgreSQL

TMS — Transport Aggregator

System Design & Architecture

Version: 1.0 **Audience:** Engineering team **Status:** Living document

---

1. Overview

TMS is a **Transportation Management System for Indian 3PL transport aggregators**. The company does not own a fleet; it procures vehicles from the open market (fleet owners and brokers), manages the resulting documentation and money flow, bills clients, and pays vendors while tracking its margin.

The system is a single **Next.js 14 (App Router) full-stack application** with a relational database, role-based portals, an operational workflow engine for indents, document management, billing, analytics and a notification outbox.

1.1 Goals

- Digitise the end-to-end order-to-cash flow: client indent -> vendor allocation -> trip execution -> POD -> billing -> payments.
- Give clients, vendors and drivers self-service portals with KYC onboarding and document upload.
- Provide compliance-aware operations (GST invoices, e-way bill flags, document expiry alerts).
- Be deployable with minimal infrastructure (SQLite for dev, PostgreSQL for production).

1.2 Non-goals (current scope)

- Live GPS telematics ingestion from third-party devices (adapter stubbed).
- Full GST/e-way bill government API filing (adapter stubbed).
- Native mobile apps (responsive PWA-style driver web app instead).

---

2. Business Context

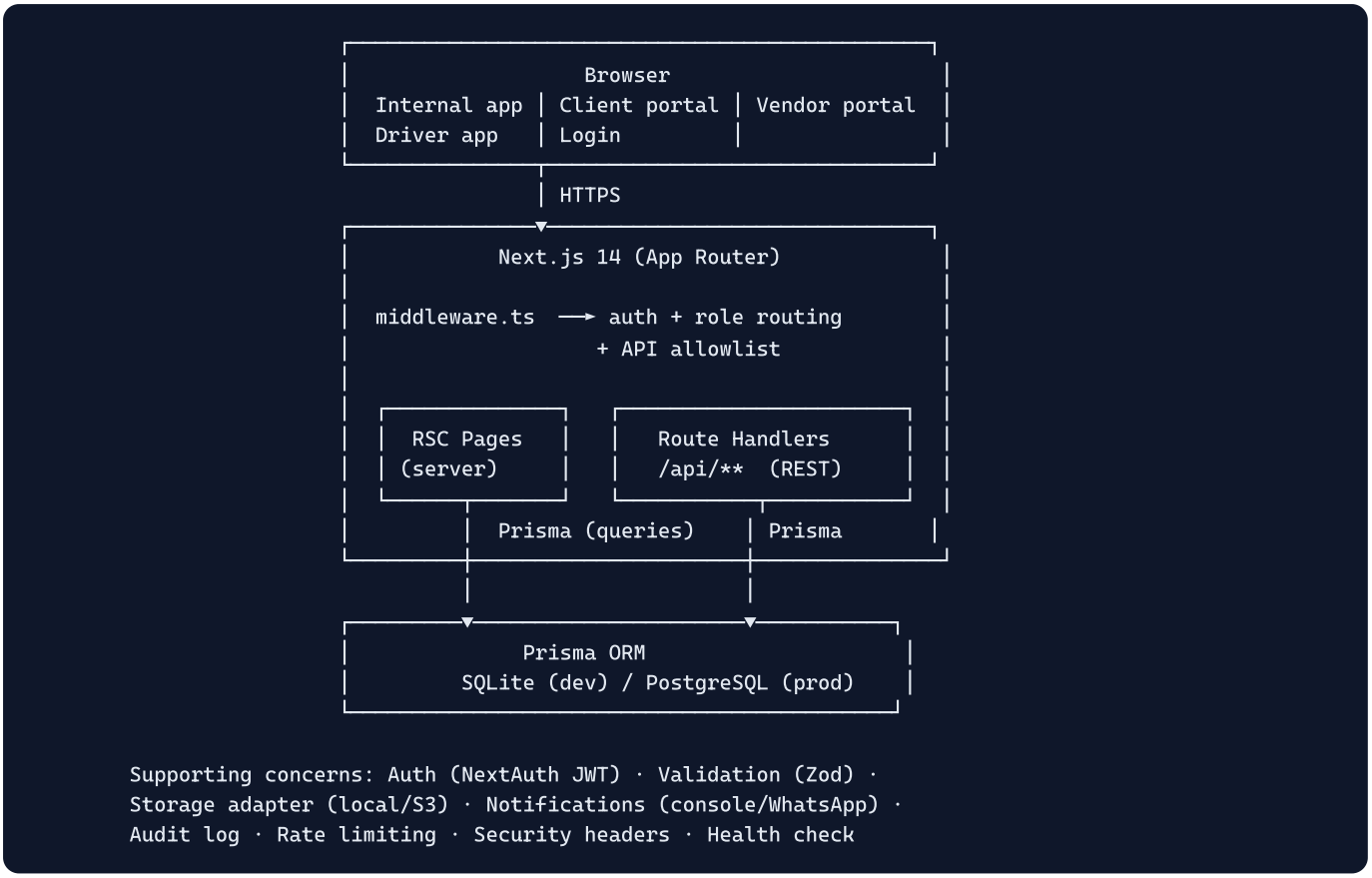
Actors

Actor	Description
Client	Raises vehicle requirements (indents), receives invoices
Vendor (Fleet Owner / Broker)	Supplies vehicles; receives payments
Driver	Operates the trip; uploads POD; shares location
Operations	Internal team: create indents, allocate, run trips
Accounts	Internal team: invoicing, payments
Admin / Senior	Full access; approves onboarding and indent advances

Money flow: Client Rate - Vendor Rate = **Margin** (per trip/indent). Advances are paid to vendors before POD; balance after POD.

---

3. High-Level Architecture



3.1 Rendering & data strategy

- **Reads** are done in **React Server Components** calling Prisma **directly** (no internal HTTP hop) for performance and simplicity.
- **Writes** go through **Route Handlers** (`/api/**`) from client components (forms), which enforce auth, validation and audit.
- Lists are server-rendered; interactive tables/boards are client components receiving serialized rows.

4. Tech Stack

Layer	Choice
Framework	Next.js 14 (App Router, RSC)
Language	TypeScript
UI	Tailwind CSS + custom shadcn-style primitives (Radix where needed)
Icons	lucide-react
ORM	Prisma
Database	SQLite (dev) / PostgreSQL (prod)
Auth	NextAuth (Credentials + JWT sessions), bcryptjs
Validation	Zod
Charts	Custom SVG donut + Recharts available
Testing	Vitest
PDF tooling	puppeteer-core + marked (docs only)
Runtime	Node.js 20+

5. Repository Structure

```
TMS/
├─ prisma/
│  ├─ schema.prisma          # data model (SQLite for dev)
│  ├─ migrations/            # SQL migrations
│  └─ seed.ts                # demo data
├─ src/
│  ├─ middleware.ts          # auth, role routing, API allowlist
│  ├─ app/
│  │  ├─ (dashboard)/        # internal app (ADMIN/OPS/ACCOUNTS)
│  │  ├─ portal/client/      # client portal
│  │  ├─ portal/vendor/      # vendor portal
│  │  ├─ driver/             # driver (mobile) app
│  │  ├─ onboard/            # self-onboarding (pre-approval gate)
│  │  ├─ api/                # route handlers (REST)
│  │  └─ files/[...path]/    # authenticated document streaming
│  ├─ components/
│  │  ├─ ui/                 # primitives (button, card, input, badge…)
│  │  ├─ forms/              # feature forms (client/vendor/indent…)
│  │  ├─ app-shell.tsx       # sidebar + topbar + command palette
│  │  ├─ data-table.tsx      # reusable search/sort/paginate table
│  │  ├─ kanban.tsx          # native DnD board
│  │  └─ donut.tsx, toast.tsx, command-palette.tsx …
│  └─ lib/
│     ├─ prisma.ts           # Prisma singleton
│     ├─ auth.ts             # NextAuth config + getSessionUser
│     ├─ api.ts              # HTTP helpers (ok/created/badRequest…)
│     ├─ queries.ts          # all read queries (server)
│     ├─ validation.ts       # Zod schemas + validate()
│     ├─ logic.ts            # pure business logic (tested)
│     ├─ audit.ts            # audit logging
│     ├─ notifications.ts    # outbox creation on status change
│     ├─ notify.ts           # provider adapter + processOutbox
│     ├─ storage.ts          # storage adapter (local/S3)
│     ├─ sequence.ts         # human-readable number generation
│     └─ company.ts          # company profile (invoice header)
├─ public/uploads/           # local document storage (gitignored)
├─ Dockerfile / docker-compose.yml / DEPLOYMENT.md
└─ README.md / ARCHITECTURE.md
```

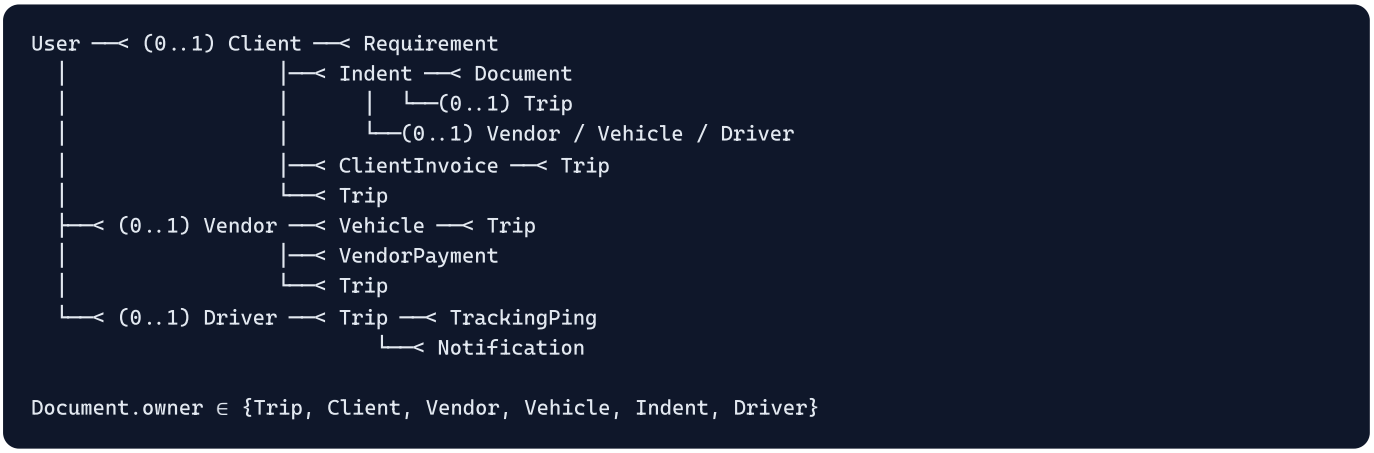
6. Data Model

> Enums are stored as strings for SQLite/Postgres portability.

6.1 Entities

Model	Purpose	Key fields
User	Login + role + link to client/vendor/driver	email, password(hash), role, clientId/vendorId/driverId
Client	Customer master (+ onboarding status, KYC, bank)	name, gstin/pan, addresses, onboardingStatus
Vendor	Fleet owner / broker (+ onboarding status, bank)	name, type, gstin/pan, bank*, onboardingStatus
Vehicle	Vehicle master (+ docs, telematics)	number, type, rc/insurance/fitness/puc/permit
Driver	Driver master (+ onboarding status)	name, phone, license*, onboardingStatus
Indent	Client requirement + full workflow	stage, consignor/consignee, material, freight, advance/balance
Requirement	Lightweight client vehicle request	reqNumber, route, qty, clientRate, status
Trip	Executed movement (LR)	lrNumber, clientRate, vendorRate, margin, status
Document	Uploaded file attached to any owner	tripId/clientId/vendorId/vehicleId/indentId/driverId, type, url
ClientInvoice	GST invoice to client	invoiceNumber, subTotal, gstAmount, totalAmount, status
VendorPayment	Payment to vendor	amount, advanceAmount, tdsAmount, netPayable, status
Advance	Ad-hoc advance record	vendorId/driverId/tripId, amount
TrackingPing	GPS ping per trip	tripId, lat, lng, note
Notification	Message outbox	channel, to, body, tripId, status
AuditLog	Immutable action log	userId, action, entity, entityId, meta

6.2 Entity relationships (simplified ERD)



6.3 Indent state machine



- `approvalStatus` : NOT_SUBMITTED | PENDING | APPROVED | REJECTED
- Only **Admin** can move to APPROVED/REJECTED.
- Advance is blocked until approved (guard in `/api/indents/[id]/workflow`).

6.4 Trip lifecycle

```
PENDING ─► DISPATCHED ─► IN_TRANSIT ─► DELIVERED ─► POD_RECEIVED ─► INVOICED ─► CLOSED
                                                └──────────────► CANCELLED
```

Status changes may create a **Notification** and an **AuditLog** entry.

6.5 Onboarding approval (Client/Vendor/Driver)

```
PENDING ─► SUBMITTED ─► APPROVED   (portal access granted)
                    └─► REJECTED   (re-submit allowed)
```

Portal layouts redirect non-APPROVED entities to `/onboard/<role>`.

---

7. Domain Modules & Workflows

7.1 Indent (client requirement) — the core flow

1. **Create** indent with consignor/consignee, vehicle requirement, material, freight.
2. **Upload** client invoice + vehicle documents (indent-scoped `Document`s).
3. **Submit for approval** -> senior **Approve/Reject**.
4. **Record advance** (amount, mode, reference).
5. **Mark POD received** (after POD upload).
6. **Record balance** -> COMPLETED.
7. Optionally **allocate** vendor/vehicle/driver and **convert to Trip** (generates an LR).

7.2 Trips (execution)

- Created manually or converted from an indent; auto **LR number** (`LR-<year>-####`).
- Margin computed as `clientRate − vendorRate`.
- Documents: LR, e-way bill, POD, challan, gate pass.
- Tracking pings (operator/vendor/driver) surfaced to clients.
- Status workflow with timestamps.

7.3 Billing & payments

- **ClientInvoice**: built from POD-received trips; GST (default 18%); printable GST tax invoice.
- **VendorPayment**: `net = amount − advance − TDS`; advance/TDS tracking; mark paid.
- **Reports**: lane-rate benchmarking, client profitability, vendor scorecard, outstanding ageing.

7.4 Reports & analytics

- Lane rates (avg client/vendor/margin per lane, min/max).
- Client profitability (revenue, cost, margin %, outstanding).
- Vendor scorecard (trips, completion %, margin, pending payable).
- Outstanding ageing buckets (current / 1–30 / 31–60 / 60+).

---

8. Authentication & Authorization

8.1 Authentication

- **NextAuth CredentialsProvider** with **JWT sessions**.
- Passwords hashed with **bcryptjs** (10 rounds).
- Login is **rate-limited** (8 attempts / 5 min per email) via in-memory limiter.
- Session carries `role`, `clientId`, `vendorId`, `driverId`.

8.2 Roles

Role	Home	Scope
ADMIN	`/`	Full access; approvals (onboarding, indent)
OPERATIONS	`/`	Internal operations
ACCOUNTS	`/`	Internal finance
CLIENT	`/portal/client`	Own data only
VENDOR	`/portal/vendor`	Assigned trips, own payments
DRIVER	`/driver`	Own trips only

8.3 Enforcement (defence in depth)

1. **Edge middleware** — page routing per role + **API allowlist** (portal roles get `403` on internal APIs; only `/api/portal/*` + specific endpoints are allowed).
2. **Server layouts** — re-check session and onboarding status; redirect.
3. **Route handlers** — `requireAuth()`; ownership checks (e.g. vendor/driver document upload verified against the trip).
4. **Ownership scoping** — client invoice detail verified against session `clientId`.

8.4 Middleware API allowlist (summary)

```
CLIENT : POST /api/requirements · /api/portal/*
VENDOR : POST /api/trips/:id/(documents|tracking) · /api/portal/*
DRIVER : POST /api/trips/:id/(documents|tracking) · PATCH /api/trips/:id · /api/portal/*
Internal roles : all APIs
Public : /api/auth/*, /api/health, /files/* (auth required), /login
```

9. API Reference

All endpoints require an authenticated session unless noted.

9.1 Master data

Method	Endpoint	Description
GET/POST	`/api/clients`	list / create
GET/PATCH/DELETE	`/api/clients/[id]`	detail / update / delete
GET/POST	`/api/vendors`	list / create
GET/PATCH/DELETE	`/api/vendors/[id]`	detail / update / delete
GET/POST	`/api/vehicles`	list / create
GET/PATCH/DELETE	`/api/vehicles/[id]`	detail / update / delete
GET/POST	`/api/drivers`	list / create
GET/PATCH/DELETE	`/api/drivers/[id]`	detail / update / delete

9.2 Indents & requirements

Method	Endpoint	Description
GET/POST	`/api/indents`	list / create
GET/PATCH	`/api/indents/[id]`	detail / update + allocate
POST	`/api/indents/[id]/workflow`	submit / approve / reject / advance / pod / balance
POST	`/api/indents/[id]/stage`	set stage (board drag-drop; Admin for approve/reject)
POST	`/api/indents/[id]/convert`	convert indent -> trip (LR)
GET/POST	`/api/requirements`	list / create
GET/PATCH	`/api/requirements/[id]`	detail / update

9.3 Trips

Method	Endpoint	Description
GET/POST	`/api/trips`	list / create (auto LR + margin)
GET/PATCH	`/api/trips/[id]`	detail / status updates
GET/POST	`/api/trips/[id]/documents`	list / upload documents
GET/POST	`/api/trips/[id]/tracking`	list / add GPS pings

9.4 Finance

Method	Endpoint	Description
GET/POST	`/api/invoices`	list / create GST invoice from trips
GET/PATCH	`/api/invoices/[id]`	detail / mark paid
GET/POST	`/api/payments`	list / create vendor payment
PATCH	`/api/payments/[id]`	update / mark paid

9.5 Documents, onboarding, portals

Method	Endpoint	Description
GET/POST	`/api/documents`	list / upload documents (any owner)
DELETE	`/api/documents/[id]`	delete document + file
POST	`/api/onboarding/[type]/[id]`	approve/reject onboarding (Admin)
PATCH	`/api/portal/profile`	self-update own profile
GET/POST	`/api/portal/documents`	self documents
POST	`/api/portal/onboarding`	submit onboarding for approval

9.6 Ops & misc

Method	Endpoint	Description
GET	`/api/dashboard`	aggregated KPIs
GET	`/api/export/[entity]`	CSV export (clients, trips, invoices, payments, indents…)
GET/POST	`/api/users`, PATCH/DELETE `/api/users/[id]`	user management (Admin)
GET/PATCH	`/api/notifications/[id]`	outbox item status
POST	`/api/notifications/process`	send pending outbox
GET	`/api/health`	health probe (public)
GET	`/files/[...path]`	authenticated document streaming

10. Request / Data Flow

10.1 Authenticated mutation (example: create trip)

```
Client form ─POST /api/trips─► middleware (auth + role allowlist)
   └─► route handler: requireAuth → Zod validate(tripSchema)
        ├─ nextNumber("trip","LR")     (sequence)
        ├─ prisma.trip.create( … )     (margin computed)
        ├─ update requirement status   (allocation rollup)
        ├─ logAudit( … )
        └─ 201 JSON ─► client router.refresh()
```

10.2 Portal onboarding gate

```
Login (CLIENT) ─► middleware allows /portal/client
   └─► portal layout: getSessionUser → getOnboardingStatus
        └─ if ≠ APPROVED → redirect /onboard/client
             └─ fill form (PATCH /api/portal/profile)
               + upload docs (POST /api/portal/documents)
               + submit (POST /api/portal/onboarding)  → status SUBMITTED
        Admin approves (POST /api/onboarding/client/:id) → APPROVED
             └─ next request: portal dashboard accessible
```

10.3 Document upload & serving

```
POST /api/documents (multipart) ─► putObject(key, bytes)  [storage adapter]
     └─ writes public/uploads/<ownerType>/<ownerId>/<file>, returns /files/<key>
DB Document.url = /files/<key>
GET /files/<key> ─► requireAuth → stream file (inline, typed)
```

11. File Storage & Document Management

- **Adapter:** `src/lib/storage.ts` — `putObject` / `removeObject`.
  - `local` (default): files under `public/uploads/…`, served via authenticated `/files/[...path]`.
  - `s3` (stub): pluggable; requires `@aws-sdk/client-s3` + bucket config.
- **Why not serve `/public` directly?** `next start` does not reliably serve files added at runtime; a dedicated authenticated route guarantees availability and access control.
- **Document types:** `GST, PAN, AADHAAR, MSME, CIN, BANK_PROOF, AGREEMENT, LICENSE, RC, INSURANCE, FITNESS, PUC, PERMIT, CLIENT_INVOICE, VEHICLE_DOC, APPROVAL, LR, EWAY_BILL, POD, DELIVERY_CHALLAN, GATE_PASS, INVOICE, OTHER`.
- **Limits:** max 10 MB per file.

12. Notifications

- On trip status change, `notifyClientOfTripStatus` queues a `Notification` row.
- **Provider adapter** (`src/lib/notify.ts`): `console` (default) or `whatsapp_cloud` (env-gated Meta Cloud API).
- `processOutbox()` sends pending messages and updates status (SENT/FAILED).
- Admin outbox UI at `/notifications` with **Process Outbox**.

13. Validation & Error Handling

- **Zod schemas** per entity in `src/lib/validation.ts`; `validate(schema, data)` returns typed output or a field-level error string.
- Route handlers use HTTP helpers from `src/lib/api.ts`: `ok`, `created`, `badRequest`, `unauthorized`, `forbidden`, `notFound`, `serverError`.
- Client forms surface API error messages inline.

14. Security

Control	Implementation
Password storage	bcryptjs (10 rounds)
Sessions	NextAuth JWT; `NEXTAUTH_SECRET`
Login throttling	in-memory fixed-window rate limiter
Security headers	X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, no x-powered-by
RBAC	middleware + layout guards + per-route ownership checks
File access	authenticated `/files` route; path traversal guarded
Audit	`AuditLog` for financial/administrative actions
Health	public `/api/health`

Production hardening checklist: strong `NEXTAUTH_SECRET`, HTTPS, DB network restriction, S3 private bucket + signed URLs, provider WAF, rotate seeded demo passwords.

15. Frontend / UI Architecture

- **App Router** with route groups: `(dashboard)` (internal), `portal/*`, `driver`, `onboard`.
- **App shell** (`components/app-shell.tsx`): grouped collapsible sidebar, topbar (breadcrumbs, global search, theme toggle, notification bell, user menu with sign-out), **Command Palette (⌘/Ctrl+K)**.
- **Design system:** token-based (`bg-card`, `text-foreground`, `border-input`, etc.) with **dark mode** (`class` strategy + no-flash script). Primitives in `components/ui/*`.
- **DataTable** (`components/data-table.tsx`): reusable client-side search, sort, pagination, column show/hide, CSV export; declarative column kinds (`text|money|date|link|badge`).
- **Kanban** (`components/kanban.tsx`): native HTML5 drag-and-drop for Indents (by stage) and Trips (by status), optimistic updates + toasts.
- **Toasts** (`components/toast.tsx`) via context provider.
- **Charts:** custom SVG donut (`components/donut.tsx`).

16. Configuration & Environment

Variable	Required	Description
`DATABASE_URL`	yes	`file:./dev.db` (SQLite) or Postgres URL
`NEXTAUTH_URL`	yes	Public base URL
`NEXTAUTH_SECRET`	yes	Session signing secret
`NOTIFY_PROVIDER`	no	`console` (default) / `whatsapp_cloud`
`WHATSAPP_TOKEN`, `WHATSAPP_PHONE_ID`	if WhatsApp	Meta Cloud API creds
`STORAGE_DRIVER`	no	`local` (default) / `s3`
`S3_*`, `AWS_*`	if S3	Object storage config

17. Local Development

```
npm install
npm run db:generate      # prisma client
npm run db:migrate       # create/update SQLite schema
npm run db:seed          # demo data + users
npm run dev              # http://localhost:3000
```

> If port 3000 is taken: `$env:PORT=3210; $env:NEXTAUTH_URL="http://localhost:3210"; npm run dev`

Demo credentials (password `admin123`): `admin@tms.local`, `ops@tms.local`, `accounts@tms.local`, `client@tata.local`, `vendor@sharma.local`, `driver@tms.local`, `pending@tms.local` (self-onboarding demo).

Scripts: `dev`, `build`, `start`, `lint`, `test`, `db:generate`, `db:migrate`, `db:migrate:deploy`, `db:seed`, `db:studio`.

18. Database Migrations & Environments

- **Dev:** SQLite (`provider = "sqlite"`), migrations applied with `prisma migrate dev`.
- **Production:** PostgreSQL — switch `provider` to `"postgresql"`, set `DATABASE_URL`, then `prisma migrate dev --name init_postgres` on an empty DB and `prisma migrate deploy`.
- Enum values are stored as **strings** so the same model works on both engines.
- SQLite is suitable for a **single** app instance; use Postgres for horizontal scale.

19. Deployment

19.1 Docker (single instance, SQLite on volumes)

```
docker compose up -d --build
docker compose exec app npm run db:seed    # first run only
```

Persistent volumes: `tms_db` (`/app/prisma`), `tms_uploads` (`/app/public/uploads`).

19.2 Node on a VM

```
npm ci && npx prisma generate && npm run build
npx prisma migrate deploy
NODE_ENV=production npm run start        # behind Nginx/Caddy + TLS
```

19.3 Health probe

`GET /api/health` -> `200 {status:"ok",db:"up"}` or `503 {status:"degraded"}`.

20. Testing & Quality

- **Vitest** unit tests for pure logic (`src/lib/logic.test.ts`, `ratelimit.test.ts`): margin, ageing buckets, vendor scoring, CSV escaping, notification mapping, rate limiter.
- `npm test` runs the suite; `npm run build` performs type-check + lint + production build.
- **Smoke tests** (manual/CI) exercise auth, RBAC `403`s, workflow transitions, uploads and document serving.

21. Observability & Auditing

- **AuditLog** records create/update/delete, trip/indent status, invoice/payment actions, user management, outbox processing.
- Admin audit viewer at `/audit`.
- Server-side errors are logged (`console.error`) in `serverError()`.
- **Recommendation (future):** structured logging + correlation IDs, metrics (Prometheus), error tracking (Sentry).

22. Extension Points & Roadmap

Already stubbed / adapter-ready: notifications provider, storage provider, health, audit, CSV export.

Next phases (proposed):

1. Commercial: rate cards/contracts, fuel surcharge, detention; auto-pricing & bill reconciliation.
2. Visibility: ePOD (OTP/photo/geo), live GPS telematics + map, geofence, ETA.
3. Compliance: e-way bill + GST e-invoicing (GSP/NIC APIs).
4. Freight Audit & Payment: reconciliation, disputes, payment runs, ledgers, accounting export (Tally/Zoho).
5. Integrations: WhatsApp/SMS provider, public API + webhooks.
6. Load marketplace / spot bidding; route optimisation & backhaul.
7. Fleet maintenance (service/tyre/fuel/FASTag), OCR document capture, multi-branch scoping.

23. Key Design Decisions (ADR summary)

#	Decision	Rationale
1	Next.js full-stack (RSC + route handlers)	Single deployable, fast reads, simple ops
2	Prisma with SQLite dev / Postgres prod	Zero-config dev; scalable prod; string enums for portability
3	Server-direct reads, API for writes	Avoids internal HTTP; keeps one validated write path
4	Edge middleware allowlist for portals	Strong, centralised RBAC for distributed roles
5	Authenticated `/files` route	Runtime uploads not reliably served from `/public`
6	Adapters for notify/storage	Swap providers without touching domain code
7	Manual GPS pings + driver web app	Ship visibility without device integrations
8	Audit log as first-class entity	Financial traceability & compliance

24. Glossary

Term	Meaning
Indent	Client's vehicle requirement / order
LR	Lorry Receipt (consignment note)
POD	Proof of Delivery
e-way bill	GST electronic movement document
TBB	To Be Billed
FTL / PTL	Full Truck Load / Part Truck Load
TDS	Tax Deducted at Source
Margin	Client rate - vendor rate
Onboarding	KYC approval flow before portal access

End of document.